



INSA ROUEN NORMANDIE

Département Informatique

PROJET INTÉGRATIF SMART ROBOT

Robot Autonome

Équipe projet :

Delacroix-Henrion Mathieu

Diallo Alioune

Malherbe Ylann

Mathieu Ilan

Olivier Neil

Schetrit Jahnis

ITI3 — Année 2025-2026

Décembre 2025

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Problématique	2
2	Partie électronique	2
2.1	Cahier des charges	2
2.2	Choix des composants	2
2.2.1	Capteur suiveur de ligne	3
2.2.2	Capteur RGB	3
2.2.3	Moteur Driver	3
2.2.4	Ecran LCD	4
2.2.5	Buzzer	5
2.3	Conception du Robot	5
2.3.1	Schéma électrique	5
2.3.2	Choix techniques	6
2.3.3	Estimation du robot	7
2.4	Difficultés rencontrées et pistes d'amélioration	7
2.5	Logigramme de la réponse au cahier des charges	8
3	Partie informatique	10
3.1	Cycle en V	10
3.1.1	Analyse	10
3.1.2	Conception préliminaire	11
3.1.3	Conception détaillée	14
3.1.4	Code C	20
3.2	Conclusion	29
3.2.1	Répartition du travail	29
3.2.2	Conclusion du projet	29
3.2.3	Commit Git	30
4	Bibliographie	30

1 Introduction

1.1 Contexte

Dans le cadre de notre premier semestre dans le département ISIA, nous avons pour but de réaliser un *Smart Robot* capable de se déplacer dans une ville modélisée sous forme de cases. Equipé de capteurs, moteurs et d'une raspberry Pi 4, notre objectif est de construire et coder un robot capable de parcourir un itinéraire imposé tout en déterminant le plus court chemin permettant de passer par un certain nombre de points obligatoires. C'est en combinant électronique, algorithmique et développement en C, ainsi que l'utilisation de GitLab que nous allons répondre aux attentes de ce projet. Ce rapport est structuré de la manière suivante :

Dans un premier temps, nous présenterons la partie électronique contenant la description de la conception du robot avec le schéma électrique, les Choix techniques que nous avons effectués, ainsi que les branchements et problèmes que nous avons pu rencontrés.

Dans un deuxième temps, nous présenterons la partie informatique, depuis l'analyse descendante du problème, jusqu'à l'implémentation des fonctions et procédures en C. Enfin, nous concluons avec la répartition des tâches et ce que l'on a pu tirer de ce projet.

1.2 Problematique

La problématique principale de ce projet est de faire en sorte que le robot parcoure la ville de manière autonome et optimale en interprétant correctement les descriptions de la ville. En effet, le robot doit lire et traduire ces informations en une suite d'ordre (AV, TG, TD) lui permettant de se déplacer dans la ville. Celui-ci doit non seulement suivre les lignes au sol en faisant attention à corriger d'éventuelles déviations, mais aussi détecter et signaler les pastilles colorées signalant les points de passages obligatoires. En définitive, notre robot doit être capable de réaliser plusieurs actions en parallèle de manière autonome.

2 Partie électronique

2.1 Cahier des charges

Pour la partie électronique, le cahier des charges est assez clair et précis car il était renseigné dans le sujet. La partie électronique se concentre sur le robot, ses composants et son fonctionnement. Les attendus sont les suivants :

Un robot qui suit un chemin tracé au sol, passant par des points précis, et qui revient à son point de départ. Lorsque le robot passera sur le point distinguable par une pastille de couleur, il émettra un son distinct et affichera la couleur de la pastille sur le LCD. Chacune de ses actions doivent être affichée en temps réel sur l'écran du LCD (Avancer, Virage à Gauche et Virage à Droite). L'arrivée qui est également le point de départ sera visible et détectable par le robot car il sera représenté par une pastille rouge.

Notre objectif va alors être de mettre en place les composants nécessaires ainsi que coder les algorithmes indispensables afin de répondre à ce cahier des charges.

2.2 Choix des composants

Afin de réaliser notre robot, nous avons plusieurs composants à choisir tant sur le nombre de composants à intégrer dans la construction de notre robot, que sur l'emplacement dans le robot. En ce qui concerne les moteurs, ceux-ci sont alimentés à l'aide de piles connectées en parallèles et sont reliés à un interrupteur. La raspberry Pi est alimentée par une batterie externe afin de garantir une autonomie et une alimentation stable. Le buzzer et les capteurs (suiveurs de ligne et RGB) sont connectés aux GPIO de la RaspberryPi et alimentés avec 3,3V. Concernant l'écran LCD, celui-ci nécessite une alimentation de 5V comme précisé dans sa documentation

technique. Avant de monter intégralement les composants sur le robot, nous avons testés chacun des composants séparément :

- Une unité de test pour les moteurs (Avancer, reculer, s'arrêter...)
- Une unité de test pour les capteurs (détecter une surface sombre, détecter une surface claire, détecter une pastille de couleur...)
- Une unité de test pour l'affichage sur l'écran LCD (afficher 'Bonjour', effacer l'écran, afficher la couleur détectée par le capteur RGB...)

Ces tests nous ont permis d'ajuster l'emplacement des composants, la calibration des capteurs, de la même manière que trouver plus facilement les composants possiblement défectueux.

2.2.1 Capteur suiveur de ligne

Rôle

Les capteurs suiveurs de ligne permettent au robot de détecter et donc de suivre une ligne noire tracée sur une surface plus claire. L'objectif est de guider les mouvements du robot, de telle sorte que celui-ci corrige sa direction s'il n'est pas bien aligné sur la trajectoire de la ligne. En incluant ces types de capteurs dans notre projet, nous sommes capables de garder le robot sur la ligne noire, de détecter lorsque celui-ci dévie de la trajectoire, mais cela permet aussi de repérer les intersections et virages lui permettant de s'orienter.

Fonctionnement

Ces types de capteurs fonctionnent de la manière suivante :
Le noir absorbe la lumière tandis que le blanc ou les couleurs plus claires la réfléchissent. Les capteurs de ligne sont alors composés d'une LED infrarouge qui éclaire le sol en continu et d'une photodiode mesurant la lumière renvoyée. Concrètement, si un capteur suiveur de ligne se trouve sur une surface claire, alors la lumière infrarouge est fortement réfléchi. Le capteur reçoit donc beaucoup de lumière et envoie le signal indiquant qu'il s'agit d'une surface claire. Au contraire, sur une surface noire, la lumière est absorbée donc le capteur ne reçoit que très peu de lumière. Celui-ci envoie alors le signal indiquant qu'il s'agit d'une surface noire.

2.2.2 Capteur RGB

Rôle

Le capteur de couleur TCS 34725 permet au robot de détecter la couleur des pastilles présentes sur le parcours comme la pastille rouge indiquant le point de départ du robot. Ce capteur est très important car il permet d'identifier correctement les points de passage, mais aussi d'afficher la couleur que ce capteur détecte sur l'écran LCD. Il est essentiel d'inclure ce capteur, sans quoi, notre robot ne serait pas capable de repérer les points de passages obligatoires du parcours repéré par une pastille de couleur verte.

Fonctionnement

Le capteur de RGB, comme son nom l'indique, permet de repérer trois couleurs : rouge, vert et bleu. Le TCS 34725 possède une petite LED blanche pour éclairer la surface et alors mesurer l'intensité lumineuse pour chacune des composantes (composante rouge, bleue ...). Ensuite, les valeurs sont converties en valeurs numériques via le moyen de communication I2C qui sont ensuite lues par la carte embarquée de la Raspberry Pi 4. Les trois intensités sont alors combinées pour déterminer de quelle couleur il s'agit.

2.2.3 Moteur Driver

Rôle

Le moteur driver est un intermédiaire entre la Raspberry Pi 4 et les moteurs du robot car il permet de fournir assez de courant aux moteurs, régler la vitesse, contrôler le sens de rotation des roues mais aussi de protéger la raspberry car celle-ci ne peut pas alimenter directement les moteurs. En effet, la Raspberry ne fournirait que 3.3V ou au maximum 5V pour les moteurs.

tandis que ceux-ci peuvent être alimentés jusqu'à 12V. Il est donc indispensable d'utiliser un moteur Driver.

Fonctionnement

Un moteur Driver contient 2 canaux moteurs (moteur 1, moteur 2) ainsi que 6 broches de contrôle :

- Enable1 : permet de régler la vitesse du moteur 1
- Enable2 : permet de régler la vitesse du moteur 2
- IN1, IN2 : permettent de contrôler le sens du moteur 1
- IN2, IN3 : permettent de contrôler le sens du moteur 2

Afin de contrôler correctement le moteur, il est important de savoir comment fonctionnent ces différentes broches. Tout d'abord, pour contrôler la vitesse d'un moteur, nous avons besoin du PWM (Pulse Width Modulation) qui permet de simuler une tension variable à partir d'un signal (HIGH ou LOW). Lorsque l'on met cette valeur à HIGH(1), le moteur est activé à pleine vitesse. Inversement, lorsque l'on met cette valeur à LOW(0), le moteur est désactivé. Enfin, en ce qui concerne les broches permettant de contrôler le sens du moteur, celles-ci fonctionnent comme suit :

IN1	IN2	Action
0	0	rotation dans le sens 1
0	1	rotation dans le sens 2
1	0	arrêt
1	1	frein

TABLE 1 – Fonctionnement des broches

2.2.4 Ecran LCD

Rôle

Un écran LCD I2C permet d'afficher les informations provenant du robot comme les indications concernant son trajet sur le parcours (Tourner gauche, intersection. . .) Mais aussi d'afficher la couleur des pastilles situées sur les points de passages obligatoires. Concrètement, il s'agit de l'interface entre le robot et nous, utilisateurs.

Fonctionnement

Un écran LCD utilise beaucoup de fils, pour gérer cela, nous utilisons un LCD avec l'I2C pour réduire à seulement 2 fils de communication. Le câblage est alors plus simple et cette configuration est idéale pour notre robot surtout quand on utilise déjà beaucoup de GPIO avec les capteurs et moteurs.

Seulement 4 lignes sont utilisées :

- SDA : données
- SCL : horloge
- VCC : alimentation
- GND : masse

Il est intéressant de noter que tous les périphériques I2C sont connectés en parallèle sur SDA/SCL. C'est exactement ce que l'on a pu faire avec le capteur RGB. L'écran LCD fonctionne de la manière suivante :

Le micro contrôleur envoie les commandes comme le texte à afficher ou encore effacer l'écran. Le module I2C lui, reçoit les données et les convertit en signaux compréhensibles par le LCD.

2.2.5 Buzzer

Rôle

Le Buzzer a servi d’emetteur de son lorsque le robot détecte sur une pastille. Concrètement il s’agit d’un moyen pour le robot de nous confirmer qu’il a bien repéré la pastille.

Fonctionnement

Le buzzer est un composant assez simple qui ne demande pas de protocole de communication particulier. Pour le gérer, nous utilisons directement un GPIO de la Raspberry Pi 4. Le câblage est rapide et cette configuration est pratique car Seulement 3 lignes sont utilisées sur le module :

- S : Signal de commande
- VCC : L’alimentation
- GND : La masse

2.3 Conception du Robot

2.3.1 Schéma électrique

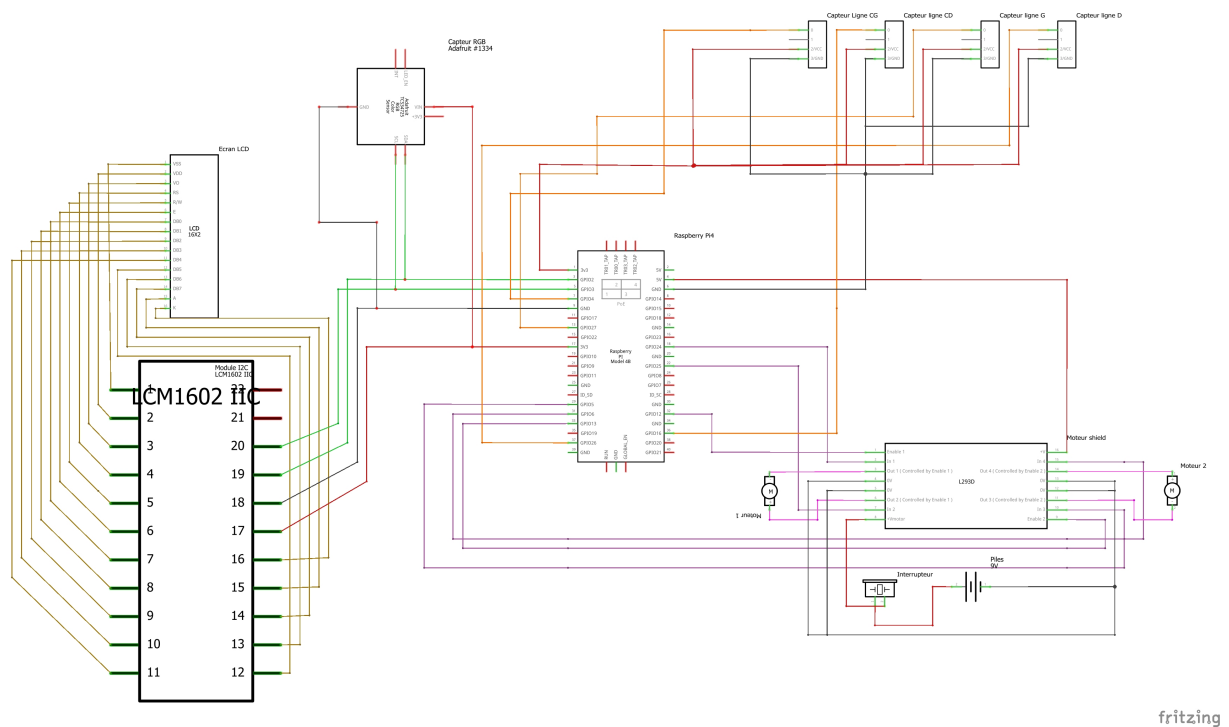
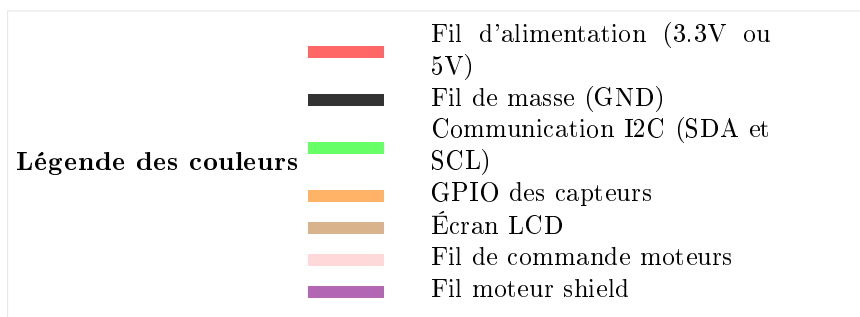


FIGURE 1 – Schéma électrique du robot autonome



2.3.2 Choix techniques

Pour le montage du robot, nous avons commencé par nous rassembler et discuter des différents composants que nous pouvions inclure au robot. Tout d'abord, nous avons discuté du meilleur emplacement pour les capteurs de contrastes, essentiels au projet pour détecter les lignes noires du parcours. Afin de suivre au mieux la ligne, trois possibilités s'offraient à nous :

- Deux capteurs repérant la ligne noire devant
- Deux capteurs de part et d'autre de la ligne noire devant
- Trois capteurs, un repérant la ligne noire et deux capteurs détectant une surface autre que noire

La première option nous semblait au début la meilleure, mais lors des tests nous avons remarqué que les deux capteurs quittaient vite la ligne noire et le robot était alors "perdu". L'option à trois capteurs nous semblait très fortement similaire à la deuxième car la correction de trajectoire s'effectuait à l'aide des capteurs autour de la ligne. Ces raisons nous ont donc poussé à choisir la solution avec deux capteurs autour de la ligne renvoyant l'information de la déviation permettant au programme de la corriger. Les tests furent après cela, très concluants.

De plus, nous avons choisi de placer deux capteurs suiveurs de lignes sur les côtés, un capteur sur le côté droit, ainsi qu'un capteur sur le côté gauche afin de détecter les intersections et virages. Les capteurs se situent assez proche des roues, ce qui signifie que le robot se retrouvera au centre de l'intersection lorsque celui-ci devra effectuer un virage. Pour cette raison, nous avons choisi d'implémenter un virage sur place plutôt que en avançant. Mais encore, le virage s'arrête lorsque le capteur de ligne (et non d'intersection) externe au virage retrouve une ligne noire. Cela est alors possible puisque nous avons choisi d'interdire les demis-tours.

En ce qui concerne le capteur de couleur, étant donné que les pastilles étaient au sol, nous l'avons logiquement positionné au plus près, en dessous du robot.

Enfin nous avons calibré chaque composant pour que le rendu soit le plus propre possible et le plus près du sujet. Pour ce qui est de la partie moteur, le courant est contrôlé par un interrupteur et le reste pas un "Moteur Driver". Lors du parcours le robot devra bipper (avec notre buzzer) lorsqu'il rencontre une pastille. et s'il croise la pastille rouge alors il devra s'arrêter.

A la suite de tous ces choix, notre robot a alors pu être construit et décoré, prêt à être utilisé.

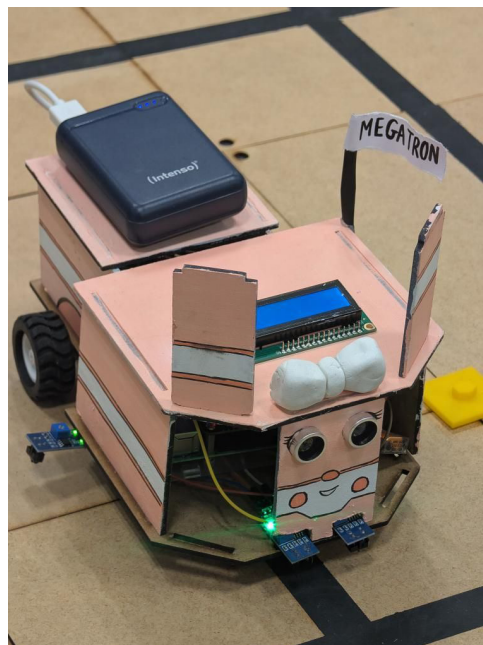


FIGURE 2 – smart robot personnalisé

2.3.3 Estimation du robot

Afin d'estimer les différentes valeurs de puissances consommées par notre robot nous sommes allée chercher les valeurs dans les datasheets associées aux composants sur leurs différents sites de vente ou sur le site du producteur. Si certaines datasheets sont indisponibles ou introuvables, alors, nous avons pris la décision de choisir une valeur moyenne des produits similaires que nous pouvons trouver.

Appareil	Tension (V)	Courant (mA)	Puissance (W)
TCS34725	3,3	0,33	$1,09 \times 10^{-3}$
Capteur de ligne x4	3,3	20	0,264
L293D x2	3,3	600	3,96
LCD1602	5	21	0,105
Buzzer actif	3,3	30	0,099
PCF8574	3,3	0,04	$0,132 \times 10^{-3}$
Raspberry Pi	5	1 200	6

TABLE 2 – Bilan de consommation du robot

Ainsi pour estimer l'autonomie de notre batterie nous allons simplement venir diviser la capacité de celle-ci par le courant total consommé par les différents composants.

Les composants liés à la batterie sont :

- Le buzzer actif
- Les 4 capteurs suiveurs de lignes
- Le capteur RGB
- Les 2 contrôleurs de pilote du moteur
- L'écran LCD
- La Raspberry Pi 4

Ainsi la consommation estimée des composants est d'environ 10.4 W, dont 4.32 W pour les éléments alimentés en 3,3 V et 6.11 W pour ceux alimentés en 5 V.

Il est important de noter que les moteurs, étant alimentés séparément par des piles, leur consommation n'est pas incluse dans ce calcul.

L'autonomie du système peut donc être estimée à partir de l'énergie fournie par la batterie. Elle est donnée par le rapport entre l'énergie stockée dans la batterie et la puissance consommée par le système (environ 10.4 W hors moteurs). Ainsi, pour une batterie d'énergie 37.0 (en Wh), l'autonomie est approximativement $T = 37.0 / 10.4 = 3\text{h}33 \text{ min}$. En tenant compte des pertes liées aux convertisseurs de tension, l'autonomie réelle est comprise entre 80 % et 90 % de cette valeur théorique.

2.4 Difficultés rencontrées et pistes d'amélioration

Nous avons rencontré de nombreuses difficultés, aussi bien sur le plan électronique que sur le plan organisationnel.

Tout d'abord, pour ce qui était de l'électronique, les premiers problèmes furent de s'accoutumer avec les composants. Pour la majorité des composants, on ne les avait jamais vus avant et comprendre leur fonctionnement nécessitait d'avoir un code lié et de pouvoir faire des tests. Or nous n'avions pas de code pré-disposé. La suite fut de passer de la théorie à la pratique. En effet, lorsque nous avons codé le comportement du robot, nous nous sommes rendu compte de

la différence entre notre comportement théorique et le fonctionnement réel. Une erreur dans la récupération de l'information des capteurs nous empêchait d'avancer.

Un autre problème qui nous a beaucoup occupé fut la détection de couleur. Lors de nos tests préliminaires le capteur fonctionnait comme souhaité, mais en pratique non, et notre manière de trouver les bonnes valeurs pour le faire fonctionner restait assez hasardeuse.

De plus, faire un montage stable pour le robot fut une réelle épreuve. Certains câbles ne se branchaient pas correctement et/ou se détachaient lorsque nous faisions rouler le robot, nous faisant passer parfois beaucoup de temps à reperer le problème. Faire tenir les composants sur la structure du robot n'était pas de tout repos non plus. Il était difficile de monter le robot de manière stable dû à ses dimensions assez petites.

De l'autre coté nous avons aussi eu des difficultés d'organisation. Au début, nous avons séparé le groupe en deux. Mais très vite, nous avons remarqué que nous perdions du temps et de l'efficacité. Alors nous avons choisi de faire trois groupe de deux ce qui nous a permis de prêter plus de temps à la partie algorithmique tout en restant efficace sur la partie électronique.

Il va sans dire que notre robot reste largement améliorable tant dans notre gestion des tâches et de l'effectif que dans notre relecture du code et notre vigilance sur ce que l'on écrit. La détection de couleur pourrait être plus stable et efficace. Après de nombreux essais, le robot échoue parfois à accomplir sa tâche comme nous avons pu en faire les frais lors de la soutenance. Seulement 2 pastilles sur les 3 ont été détectée. Cela est dû à plusieurs paramètres comme l'éclairage de la pièce, ou encore la couleur du circuit.

2.5 Logigramme de la réponse au cahier des charges

A la suite de toutes ces explications, choix, et prises de décisions, nous avons pu répondre au cahier des charges imposé. Afin d'illustrer cela, nous avons réalisé un Logigramme du robot reprenant chacune des parties du cahier des charges et la réponse par laquelle nous avons répondu à celui-ci.

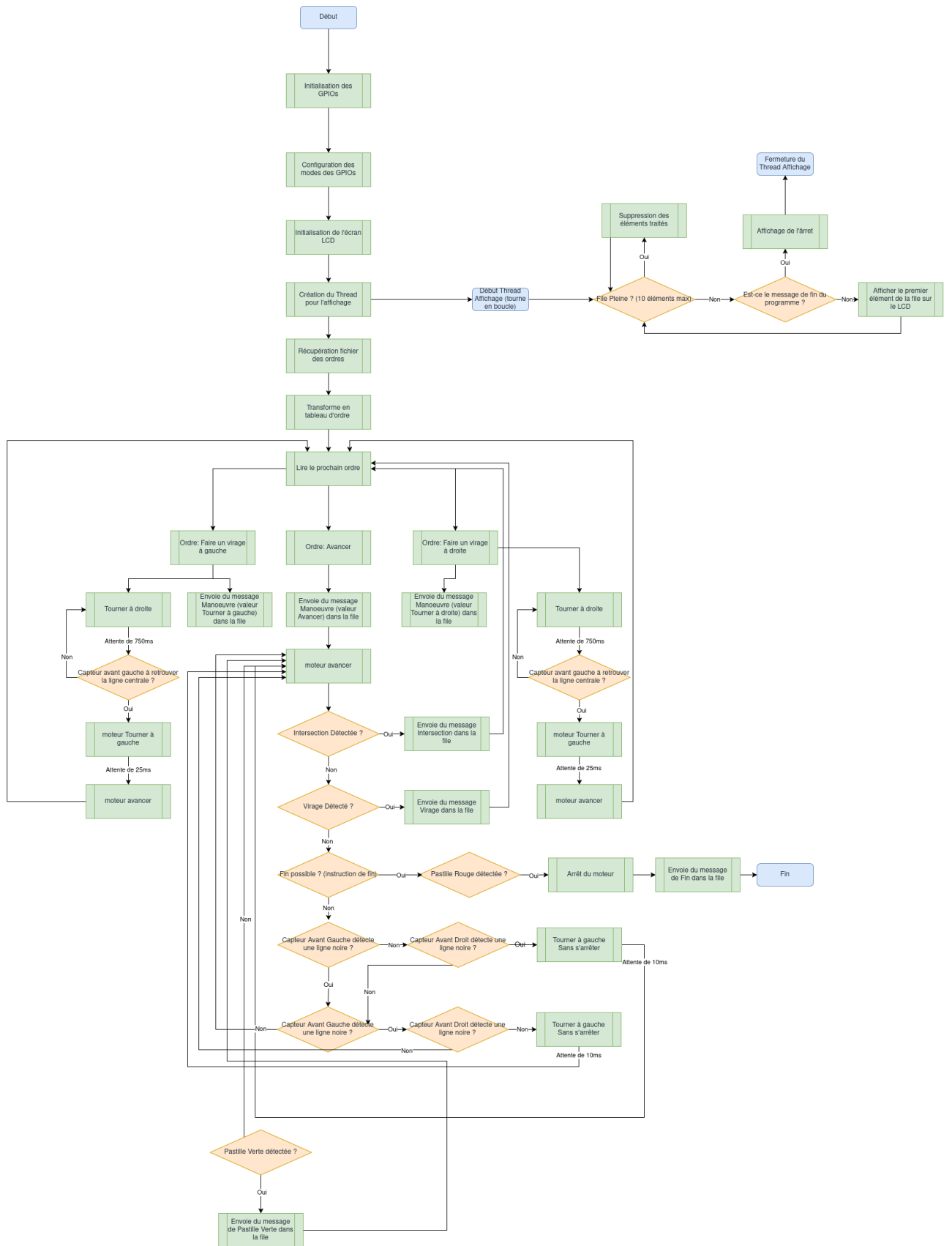


FIGURE 3 – Logigramme du robot

B. Types

En réalisant cette analyse descendante, nous nous sommes rendus compte que nous aurions besoin de plusieurs types afin de mener à bien l'algorithme souhaité. Voici les Types que nous souhaitons mettre en place par la suite :

Type Direction : Ce type permet d'associer une direction(Sud, Nord, Ouest, Est) au robot et permet donc de faciliter l'interdiction des demi-tours quant à la direction actuelle du robot.

Type Ordre : Ce type est une énumération qui permet de donner au robot les actions qu'il peut effectuer tout au long du parcours (Avancer, Tourner Droite, Tourner Gauche).

Type Robot : Ce type permettra de regrouper les informations du robot comme la case ou il se trouve et sa direction.

Type Chemin : Ce type permettra de contenir l'ensemble des points par lequel le robot doit passer pour réaliser le chemin le plus court.

Type EtatDuDijkstra : Ce type permettra de renseigner le point actuel ou l'on se trouve, le point depuis lequel on arrive (point précédent) ainsi que le chemin pour parvenir jusqu'au point actuel.

Type Solution : Ce type sera utilisé lors de la résolution du chemin le plus court, regroupant alors l'ordre dans lequel nous allons passer par les points obligatoires ainsi que le chemin complet des points par lesquels nous allons passer.

Type File : Le dernier type est essentiel car nous avons choisi d'utiliser des files pour l'algorithme de Dijkstra. Cette File stockera les différents états du Dijkstra que l'on pourra obtenir grâce aux indice de début et fin de la file.

Pour ce qui est du Graphe, nous utilisons les TAD déjà implémentés de Monsieur Delestre utilisant de la même manière les TAD ListeChaineListe ainsi que tableHachage.

3.1.2 Conception préliminaire

A. TAD

TAD tDirection

Nom : tDirection

Opérations :

- **directionOuest** : tDirection $\rightarrow$ tDirection
- **directionEst** : tDirection $\rightarrow$ tDirection

TAD Robot

Nom : Robot

Utilise :

- tDirection
- NaturelNonNul

Opérations :

- **setDirection** : tDirection $\times$ Robot $\rightarrow$ Robot
- **getDirection** : Robot $\rightarrow$ tDirection
- **getCaseRobot** : Robot $\rightarrow$ NaturelNonNul
- **setCaseRobot** : Robot $\times$ NaturelNonNul $\rightarrow$ Robot

TAD Chemin

Nom : Chemin

Utilise :

— NaturelNonNul

Opérations :

— **obtenirChemin** : Chemin $\rightarrow$ tableau[1.. NB_POINTS_MAX] de NaturelNonNul

— **obtenirNbPoints** : Chemin $\rightarrow$ NaturelNonNul

— **fixerNbPoints** : Chemin $\times$ NaturelNonNul $\rightarrow$ Chemin

TAD File

Nom : File

Utilise :

— EtatDuDijkstra

— Naturel

Opérations :

— **initialisation_file** : $\rightarrow$ File

— **estVideFile** : File $\rightarrow$ Naturel

— **estPleineFile** : File $\rightarrow$ Naturel

— **enfiler** : File $\times$ EtatDuDijkstra $\rightarrow$ File

— **defiler** : File $\rightarrow$ File $\times$ EtatDuDijkstra

— **liberer_file** : File $\rightarrow$

Préconditions : enfiler($f, pDij$) : non estPleineFile(f)

Préconditions : defiler(f) : non estVideFile(f)

2. Signatures des fonctions

Dans cette sous partie, nous établirons les signatures de toutes les fonctions et procédures utilisées dans la partie algorithme du projet. Toutefois, pour la partie conception détaillée, nous ne présenterons que les algorithmes non triviaux et essentiels à notre résolution du problème principal : Trouver le chemin le plus court.

Fonctions et procédures du TAD Robot

— Fonction directionOuest(direction: tDirection) : tDirection

— Fonction directionEst(direction: tDirection) : tDirection

— Fonction getDirection(r: Robot) : tDirection

— Procédure setDirection(E/S r: Robot, E direction: tDirection)

— Fonction getCaseRobot(r: Robot) : NaturelNonNul

— Fonction setCaseRobot(E/S r: Robot, E nouvelleCase: NaturelNonNul)

Fonctions et procédures du TAD Chemin

— Fonction obtenirNbPoints(c: Chemin) : naturel

— Procédure obtenirChemin(E c: Chemin, S tabPoints: tableau[1..NB_POINTS_MAX] de NaturelNonNul)

— Procédure fixerNbPoints(E/S c: Chemin, E nombre_points: NaturelNonNul)

Fonctions et procédures du TAD File

- Fonction `initialisation_file()`: File
- Fonction `estVideFile(f: File)` : naturel
- Fonction `estPleineFile(f: File)` : naturel
- Procédure `enfiler(E/S f: File, E etat: EtatDuDijkstra)`
- Fonction `defiler(f: File)`: EtatDuDijkstra
- Procédure `liberer_file(E f: File)`

Fonctions et procédures du graphe et de Dijkstra

- Procédure `recuperationInfoFichier(E fileNameVille: ChaineDeCaractères, S l, nb_liaisons, nbCasesObligatoires, caseInitRObot: NaturelNonNul, tabCasesObligatoires: tableau[1..MAX_CASES_OBLIGATOIRES], tabPourGraphe[NB_POINTS_MAX][2], orientationInitRobot: tDirection)`: File
- Fonction `creationGrapheVille(tabLiaisonsVille: tableau[1..NB_POINTS_MAX][2] de NaturelNonNul, nbLiaisons: NaturelNonNul)` : G_Graphe
- Fonction `dijkstra_avec_exclusions(g: G_Graphe, depart, arrivee, nb_a_eviter : NaturelNonNul, sommet_a_eviter : tableau[1..NB_POINTS_MAX] de NaturelNonNul)` : Chemin
- Procédure `permutation(E arr: tableau[3] de NaturelNonNul, S permutations[6][3])`
- Fonction `chemin_autorise(c: Chemin)`:
- Fonction `verifier_jonction_segment(segment1, segment2: Chemin)`: Naturel
- Fonction `resoudre_chemin_plus_court(g: G_Graphe, depart, longueur_ville: NaturelNonNul, directionInitialeRobot: tDirection, point_obligatoires: tableau[3] de NaturelNonNul)`

Fonctions et procédures du fichier d'ordres

- Fonction `ordreDuChangementDeDirection(direction, nlleDirection: tDirection)`
- Procédure `determinerOrdre(E/S robot: Robot, E caseSuivanteRobot, largeurCircuit: NaturelNonNul, S ordre1, ordre2: Ordre)`
- Procédure `determinerOrdres(E c: Chemin, grapheVille: G_Graphe, robot: Robot, largeurCircuit: NaturelNonNul, S tabOrdres: tableau[NB_POINTS_MAX] d'Ordre)`
- Procédure `creationFichierOrdres(E grapheVille: G_graphe, plusCourtChemin: Chemin, largeurCircuit: NaturelNonNul, directionInitRobot: tDirection)`

3.1.3 Conception détaillée

1. Types

Type `tDirection` = {NORD, SUD, EST, OUEST }

Type `Ordre` = {AV, TG, TD, NO }

Type `Robot` = structure **Champs** :

- `caseRobot` : Naturel Non Nul
- `direction` : `tDirection`

Type `Chemin` = structure **Champs** :

- `points` tableau[1..NB_POINTS_MAX] de `NaturelNonNul`
- `direction` : `tDirection`

Type `EtatDuDijkstra` = structure **Champs** :

- `point_actuel` : `NaturelNonNul`
- `point_precedent` : `NaturelNonNul`
- `chemin` : `Chemin`

Type `Solution` = structure **Champs** :

- `ordre` : tableau[1..3] de `NaturelNonNul`
- `chemin_complet` : `Chemin`

Type `file` = structure **Champs** :

- `elemnts` : tableau[1..MAX_FILE] de `EtatDuDijkstra`
- `debut` : `NaturelNonNul`
- `fin` : `NaturelNonNul`

Algorithme .1 : resoudreCheminPlusCourt (partie 1/2)

```
1 g : Graphe (G_Graphe), point_obligatoires : tableau[3] de NNN, depart : NNN,  
   longueur_ville : NNN, directionInitialeRobot : tDirection  
2 Sortie solution : Solution  
3 Début  
4   solution  $\leftarrow$  {0}  
5   solution.chemin_complet.nb_points  $\leftarrow$  0  
6   meilleure_distance  $\leftarrow$  99999  
7   point_cles[0]  $\leftarrow$  depart  
8   pour i  $\leftarrow$  1 à 3 faire  
9     | point_cles[i]  $\leftarrow$  point_obligatoires[i - 1]  
10  fin  
11  indices_classement  $\leftarrow$  [1, 2, 3]  
12  permutations  $\leftarrow$  permutation(indices_classement)  
13  pour p  $\leftarrow$  1 à 6 faire  
14    | parcours  $\leftarrow$  [0, permutations[p][0], permutations[p][1], permutations[p][2], 0]  
15    | distance_totale  $\leftarrow$  0  
16    | estValide  $\leftarrow$  1  
17    | segments  $\leftarrow$  tableau de 4 Chemin  
18    | pour i  $\leftarrow$  1 à 4 faire  
19      | index_debut  $\leftarrow$  parcours[i]  
20      | index_fin  $\leftarrow$  parcours[i + 1]  
21      | sommet_depart  $\leftarrow$  point_cles[index_debut]  
22      | sommet_arrivee  $\leftarrow$  point_cles[index_fin]  
23      | si i > 0 alors  
24        | sommets_a_eviter  $\leftarrow$   
25          | segments[i - 1].points[segments[i - 1].nb_points - 2]  
26          | nb_a_eviter  $\leftarrow$  1 sinon  
27            | si directionInitialeRobot = SUD alors  
28              | sommets_a_eviter  $\leftarrow$  depart - longueur_ville  
29            | sinon  
30              | fin  
31              | directionInitialeRobot = NORD  
32              | sommets_a_eviter  $\leftarrow$  depart + longueur_ville  
33              | si directionInitialeRobot = EST alors  
34                | sommets_a_eviter  $\leftarrow$  depart - 1  
35              | sinon  
36                | fin  
37              | sommets_a_eviter  $\leftarrow$  depart + 1  
38            | fin  
39            | segments[i]  $\leftarrow$   
           | dijkstra_avec_exclusions(g, sommet_depart, sommet_arrivee, sommets_a_eviter, 1)  
           | fin
```

```
1 Suite
2   si segments[i].nb_points = 0 alors
3     | estValide ← 0
4     | interrompre
5   fin
6   si ¬chemin_autorise(segments[i]) alors
7     | estValide ← 0
8     | interrompre
9   fin
10  distance_totale ← distance_totale + (segments[i].nb_points - 1)
11 Fin
12 si estValide alors
13   pour i ← 0 à 2 faire
14     | si ¬verifier_jonction_segments(segments[i], segments[i + 1]) alors
15       | estValide ← 0
16       | interrompre
17     | fin
18   fin
19 fin
20 si ¬estValide alors
21   passer à l'itération suivante
22 fin
23 si distance_totale < meilleure_distance alors
24   meilleure_distance ← distance_totale
25   pour i ← 0 à 2 faire
26     | solution.ordre[i] ← point_cles[permutations[p][i]]
27   fin
28   index ← 0
29   pour i ← 0 à 3 faire
30     | debut ← (i = 0)?0 : 1
31     | pour j ← debut à segments[i].nb_points - 1 faire
32       | si index < NB_POINTS_MAX alors
33         | solution.chemin_complet.points[index] ← segments[i].points[j]
34         | index ← index + 1
35       | fin
36     | fin
37   fin
38   solution.chemin_complet.nb_points ← index
39 fin
```

Algorithme .3 : dijkstra_avec_exclusion (partie 1/2)

Entrée : $g : G_Graphe^*$, $depart :$, $arrivee :$,
 $sommets_a_eviter : NNN^*$, $nb_a_eviter : tableau[1..$
 $NB_POINTS_MAX]$ de NNN
Sortie : $resultat : Chemin$

```
1 Début
2    $resultat : Chemin;$ 
3    $resultat \leftarrow \{0\}$   $points\_visites : tableau[NB\_POINTS\_MAX][NB\_POINTS\_MAX]$ 
   de  $NNN$ 
4   pour  $i \leftarrow 1$  to  $NB\_POINTS\_MAX$  faire
5     pour  $j \leftarrow 1$  to  $NB\_POINTS\_MAX$  faire
6        $points\_visites[i][j] \leftarrow 0$  fin
7     fin
8      $file : File;$ 
9      $file \leftarrow allouer(sizeof(File))$ 
10     $initialisation\_file(file);$ 
11     $initial : EtatDuDijkstra;$ 
12     $initial.point\_actuel \leftarrow depart$ 
13     $initial.point\_precedent \leftarrow 0$ 
14     $initial.chemin.points[0] \leftarrow depart$ 
15     $initial.chemin.nb\_points \leftarrow 1$ 
16     $enfiler(file, initial);$ 
17     $points\_visites[depart][0] \leftarrow 1$ 
18    tant que la file n'est pas vide faire
19       $etat : EtatDuDijkstra;$ 
20       $etat \leftarrow defiler(file);$ 
21      si  $etat.point\_actuel = arrivee$  alors
22         $resultat.nb\_points \leftarrow etat.chemin.nb\_points;$ 
23        pour  $i \leftarrow 1$  to  $etat.chemin.nb\_points$  faire
24           $resultat.points[i] \leftarrow etat.chemin.points[i];$ 
25        fin
26         $liberer\_file(file);$ 
27        Retourner  $resultat;$ 
28      fin
29       $voisins : LCL\_Liste;$ 
30       $voisins \leftarrow G\_obtenirSommetsAdjacents(g, etat.point\_actuel);$ 
31      pour  $i \leftarrow 1$  to  $LCL\_longueur(voisins)$  faire
32         $voisin : NNN;$ 
33         $voisin \leftarrow LCL\_element(voisins, i);$ 
34        si  $voisin = etat.point\_precedent$  alors
35          ;
36        fin
37         $est\_a\_eviter : 0;$ 
38        pour  $k \leftarrow 1$  to  $nb\_a\_eviter$  faire
39          si  $voisin = sommets\_a\_eviter[k]$  alors
40             $est\_a\_eviter \leftarrow 1;$ 
41          fin
42        fin
43        si  $est\_a\_eviter$  alors
44          fin
45      fin
```

46

Algorithme .4 : dijkstra_avec_exclusion (partie 2/2)

```
1 Suite
2   deja_dans_chemin  $\leftarrow$  0;
3   si voisin  $\neq$  arrivee alors
4       pour k  $\leftarrow$  1 to etat.chemin.nb_points faire
5           si etat.chemin.points[k] = voisin alors
6               deja_dans_chemin = 1;
7           ;
8       fin
9   fin
10 fin
11 si deja_dans_chemin alors
12     ;
13 fin
14 index_precedent : NNN
15 si etat.point_actuel = -1 alors
16     index_precedent  $\leftarrow$  0;
17 fin
18 sinon
19     index_precedent  $\leftarrow$  etat.point_actuel
20 fin
21 si points_visites[voisin][index_precedent] = 1 alors
22     ;
23 fin
24 points_visites[voisin][index_precedent]  $\leftarrow$  1
25 nouvel_etat : EtatDuDijkstra;
26 nouvel_etat.point_actuel  $\leftarrow$  voisin
27 nouvel_etat.point_precedent  $\leftarrow$  etat.point_actuel;
28 nouvel_etat.chemin.nb_points  $\leftarrow$  etat.chemin.nb_points + 1;
29 pour j  $\leftarrow$  1 to etat.chemin.nb_points faire
30     nouvel_etat.chemin.points[j]  $\leftarrow$  etat.chemin.points[j] fin
31     nouvel_etat.chemin.points[etat.chemin.nb_points]  $\leftarrow$  voisin
32     enfiler(file, nouvel_etat);
33 LCL_vider(voisins)
34 fin
35 liberer_file(file);
36 Retourner resultat;
```

Algorithme .5 : determinerOrdres

Entrée : c : Chemin, $grapheVille$: Graphe , $robot$: Robot, $largeurCircuit$: NNN

Sortie : $tabOrdre$: Tableau[1..NB_POINTS_MAX] d'ordre

```
1 Déclaration  $j, nbCaseChemin$  : NNN,  $ordre1, ordre2$  : Ordre,
2    $sommetsAdjacents$  : LCL_Liste,
3    $plusCourtChemin$  : Tableau[1..NB_POINT_MAX] de NNN
4 Début
5   obtenirChemin( $c, plusCourtChemin$ );
6    $nbCaseChemin \leftarrow obtenirNbPoints(c)$ 
7    $j \leftarrow 0$ 
8   pour  $i \leftarrow 0$  to  $nbCaseChemin - 1$  faire
9      $sommetsAdjacents \leftarrow G_{obtenirSommetsAdjacents}(grapheVille,$ 
10       $getCaseRobot(robot))$ 
11      $determinerOrdre(robot, plusCourtChemin[i + 1], largeurCircuit, ordre1, ordre2);$ 
12     si  $ordre1 == NO$  alors
13       si  $j \geq 1$  et  $tabOrdres[j - 1] == AV$  et  $LCL\_longueur(sommetsAdjacents)$ 
14          $> 2$  alors
15          $tabOrdres[j] \leftarrow ordre2$ 
16          $j \leftarrow j + 1$ 
17       fin
18       sinon si  $j == 0$  alors
19          $tabOrdres[j] \leftarrow ordre2$ 
20          $j \leftarrow j + 1$ 
21       fin
22     sinon
23        $tabOrdres[j] \leftarrow ordre1$ 
24        $j \leftarrow j + 1$ 
25        $tabOrdres[j] \leftarrow ordre2$ 
26        $j \leftarrow j + 1$ 
27     fin
28    $tabOrdres[j] \leftarrow NO$ 
29 retourner  $tabOrdre$ ;
```

Algorithme .6 : determinerOrdre

Entrée : *robot* : Robot,
 caseSuivanteRobot : Naturel,
 largeurCircuit : Naturel

Sortie : *ordre1* : Ordre,
 ordre2 : Ordre

1 Déclaration

2 *caseInitRobot* : Naturel,
3 *dirRobot* : Direction,
4 *diff* : Naturel,
5 *nlleDirection* : Direction

6 Début

```
7   caseInitRobot ← getCaseRobot(robot);  
8   dirRobot ← getDirection(robot);  
9   diff ← caseSuivanteRobot – caseInitRobot;  
10  nlleDirection : Direction;  
11  si diff = 1 alors  
12    | nlleDirection ← EST;  
13  fin  
14  sinon si diff = –1 alors  
15    | nlleDirection ← OUEST;  
16  fin  
17  sinon si diff = largeurCircuit alors  
18    | nlleDirection ← SUD;  
19  fin  
20  sinon si diff = –largeurCircuit alors  
21    | nlleDirection ← NORD;  
22  fin  
23  sinon  
24    | nlleDirection ← dirRobot;  
25  fin  
26  ordre1 ← ordreDuChangementDeDirection(dirRobot, nlleDirection);  
27  setDirection(robot, nlleDirection);  
28  ordre2 ← AV;  
29  setCaseRobot(robot, caseSuivanteRobot);
```

3.1.4 Code C

Maintenant que nous avons passé l'étape de la conception détaillée, nous pouvons désormais passer au développement des algorithmes en C.

Voici donc comment nous avons décidé de développer les algorithmes présentés lors de la conception détaillée en C.

1. resoudre_Chemin_Plus_Court

La fonction `resoudre_chemin_plus_court` permet de calculer le chemin optimal d'un point de départ à lui-même en passant par trois points obligatoire et sans demi-tours :

```
1 Solution resoudre_chemin_plus_court(G_Graphe* g, unsigned int  
   point_obligatoires[3], unsigned int depart, unsigned int  
   longueur_ville, tDirection directionInitialeRobot) {  
2
```

```

3      Solution solution = {0};
4      solution.chemin_complet.nb_points = 0;
5      unsigned int meilleure_distance = 99999;
6
7      unsigned int point_cles[4];
8      point_cles[0] = depart;
9      for(unsigned int i = 1; i < 4; i++) {
10         point_cles[i] = point_obligatoires[i - 1];
11     }
12
13     unsigned int indices_classement[3] = {1, 2, 3};
14     unsigned int permutations[6][3];
15     permutation(indices_classement, permutations);
16
17     // Tester chaque permutation
18     for (unsigned int p = 0; p < 6; p++) {
19         unsigned int parcours[5];
20         parcours[0] = 0;
21         parcours[1] = permutations[p][0];
22         parcours[2] = permutations[p][1];
23         parcours[3] = permutations[p][2];
24         parcours[4] = 0;
25
26         unsigned int distance_totale = 0;
27         unsigned int estValide = 1;
28         Chemin segments[4];
29
30         // CALCUL DES SEGMENTS AVEC EXCLUSION
31         for (unsigned int i = 0; i < 4; i++) {
32             unsigned int index_debut = parcours[i];
33             unsigned int index_fin = parcours[i + 1];
34             unsigned int sommet_depart = point_cles[index_debut];
35             unsigned int sommet_arrivee = point_cles[index_fin];
36
37             // Determiner le sommet a exclure
38             unsigned int sommets_a_eviter = 0;
39             unsigned int nb_a_eviter = 1;
40
41             if (i > 0) {
42                 Chemin segment_precedent = segments[i - 1];
43                 sommets_a_eviter = segment_precedent.points[
segment_precedent.nb_points - 2];
44                 nb_a_eviter = 1;
45             } else {
46                 if (directionInitialeRobot == SUD) {
47                     sommets_a_eviter = depart - longueur_ville;
48                 }
49                 else if (directionInitialeRobot == NORD) {
50                     sommets_a_eviter = depart + longueur_ville;
51                 }
52                 else if (directionInitialeRobot == EST) {
53                     sommets_a_eviter = depart - 1;
54                 }
55                 else {
56                     sommets_a_eviter = depart + 1;
57                 }
58             }
59

```

```

60         // Calculer le chemin avec exclusions
61         segments[i] = dijkstra_avec_exclusions(g, sommet_depart,
sommet_arrivee,
62                                                     &sommets_a_eviter,
nb_a_eviter);
63
64         if (segments[i].nb_points == 0) {
65             estValide = 0;
66             break;
67         }
68
69         // Verifier les demi-tours internes
70         if (!chemin_autorise(segments[i])) {
71             estValide = 0;
72             break;
73         }
74
75         distance_totale += (segments[i].nb_points - 1);
76     }
77
78     // Verification des jonctions
79     if (estValide) {
80         for (unsigned int i = 0; i < 3; i++) {
81             if (!verifier_jonction_segments(segments[i], segments[i
+ 1])) {
82                 estValide = 0;
83                 break;
84             }
85         }
86     }
87
88     if (!estValide) {
89         continue;
90     }
91
92     // Si cette permutation est valide et meilleure
93     if (distance_totale < meilleure_distance) {
94         meilleure_distance = distance_totale;
95
96         for(unsigned int i = 0; i < 3; i++) {
97             solution.ordre[i] = point_cles[permutations[p][i]];
98         }
99
100        // Reconstruction du chemin complet
101        unsigned int index = 0;
102
103        for(unsigned int i = 0; i < 4; i++) {
104            unsigned int debut = (i == 0) ? 0 : 1;
105
106            for(unsigned int j = debut; j < segments[i].nb_points; j
++ ) {
107                if (index < NB_POINTS_MAX) {
108                    solution.chemin_complet.points[index] = segments
[i].points[j];
109                    index++;
110                }
111            }
112        }

```

```

113         solution.chemin_complet.nb_points = index;
114     }
115 }
116
117
118     return solution;
119 }

```

Listing 1 – Fonction de résolution du chemin optimal

2. dijkstra_avec_exclusions

La fonction `dijkstra_avec_exclusions` permet de déterminer le plus court chemin entre deux points sans passer par certains points ni sans faire de demi-tours.

```

1 Chemin dijkstra_avec_exclusions(G_Graphe* g, unsigned int depart,
  unsigned int arrivee, unsigned int* sommets_a_eviter, unsigned int
  nb_a_eviter) {
2
3     Chemin resultat = {0};
4
5     unsigned int points_visites[NB_POINTS_MAX][NB_POINTS_MAX];
6     for (unsigned int i = 0; i < NB_POINTS_MAX; i++) {
7         for(unsigned int j = 0; j < NB_POINTS_MAX; j++) {
8             points_visites[i][j] = 0;
9         }
10    }
11
12    File* file = malloc(sizeof(File));
13    initialisation_file(file);
14
15    EtatDuDijkstra initial;
16    initial.point_actuel = depart;
17    initial.point_precedent = 0;
18    initial.chemin.points[0] = depart;
19    initial.chemin.nb_points = 1;
20    enfiler(file, initial);
21
22    points_visites[depart][0] = 1;
23
24    while (!estVideFile(file)) {
25        EtatDuDijkstra etat = defiler(file);
26
27        if (etat.point_actuel == arrivee) {
28            resultat.nb_points = etat.chemin.nb_points;
29            for (unsigned int i = 0; i < etat.chemin.nb_points; i++) {
30                resultat.points[i] = etat.chemin.points[i];
31            }
32            liberer_file(file);
33            return resultat;
34        }
35
36        LCL_Liste voisins = G_obtenirSommetsAdjacents(*g, etat.
        point_actuel);
37
38        for (unsigned int i = 0; i < LCL_longueur(voisins); i++) {
39            unsigned int voisin = *(unsigned int *)LCL_element(voisins,
        i);

```



```

40
41 // viter le demi-tour immédiat
42 if (voisin == etat.point_precedent) {
43     continue;
44 }
45
46 // Vérifier si ce voisin est un le point exclure
47 int est_a_eviter = 0;
48 for (unsigned int k = 0; k < nb_a_eviter; k++) {
49     if (voisin == sommets_a_eviter[k]) {
50         est_a_eviter = 1;
51         break;
52     }
53 }
54
55 if (est_a_eviter) {
56     continue;
57 }
58
59 // viter de revisiter un point d j dans le chemin
actuel
60 int deja_dans_chemin = 0;
61 if (voisin != arrivee) {
62     for (unsigned int k = 0; k < etat.chemin.nb_points; k++)
63     {
64         if (etat.chemin.points[k] == voisin) {
65             deja_dans_chemin = 1;
66             break;
67         }
68     }
69
70     if (deja_dans_chemin) {
71         continue;
72     }
73
74     unsigned int index_precedent = (etat.point_actuel == (
75     unsigned int)-1) ? 0 : etat.point_actuel;
76
77     if (points_visites[voisin][index_precedent]) {
78         continue;
79     }
80
81     points_visites[voisin][index_precedent] = 1;
82
83     EtatDuDijkstra nouvel_etat;
84     nouvel_etat.point_actuel = voisin;
85     nouvel_etat.point_precedent = etat.point_actuel;
86     nouvel_etat.chemin.nb_points = etat.chemin.nb_points + 1;
87
88     for (unsigned int j = 0; j < etat.chemin.nb_points; j++) {
89         nouvel_etat.chemin.points[j] = etat.chemin.points[j];
90     }
91     nouvel_etat.chemin.points[etat.chemin.nb_points] = voisin;
92
93     enfiler(file, nouvel_etat);
94 }
LCL_vider(&voisins);

```

```

95     }
96
97     liberer_file(file);
98     return resultat;
99 }

```

Listing 2 – Fonction dijkstra avec exclusions

3. determiner_ordres

La fonction `determiner_ordres` permet de déterminer, à partir d'un chemin, l'ensemble des ordres que doit suivre le robot afin de faire le parcours demandé.

```

1 void determinerOrdres(Ordre tabOrdres[NB_POINTS_MAX], Chemin c, G_Graphe
  grapheVille , Robot robot, unsigned int largeurCircuit) {
2
3     Ordre ordre1, ordre2;
4     unsigned int plusCourtChemin[NB_POINTS_MAX];
5     obtenirChemin(c, plusCourtChemin);
6     unsigned int nbCaseChemin = obtenirNbPoints(c);
7     unsigned int j = 0;
8
9     for (unsigned int i = 0; i < nbCaseChemin - 1; i++) {
10         LCL_Liste sommetsAdjacents = G_obtenirSommetsAdjacents(
  grapheVille, getCaseRobot(robot));
11         determinerOrdre(&robot, plusCourtChemin[i + 1], largeurCircuit,
  &ordre1, &ordre2);
12         if (ordre1 == NO) {
13             if (j >= 1 && tabOrdres[j-1] == AV && LCL_longueur(
  sommetsAdjacents) > 2) {
14                 tabOrdres[j] = ordre2;
15                 j = j + 1;
16             }
17             else if (j == 0) {
18                 tabOrdres[j] = ordre2;
19                 j = j + 1;
20             }
21         }
22         else {
23             tabOrdres[j] = ordre1;
24             j = j + 1;
25             tabOrdres[j] = ordre2;
26             j = j + 1;
27         }
28     }
29     tabOrdres[j] = NO; // Marqueur de fin des ordres
30 }

```

Listing 3 – Fonction déterminer ordres

paragraphe4. determiner_ordre

La fonction `determiner_ordre` permet de déterminer, les ordres que doit suivre le robot afin d'aller d'un point à un autre en fonction de sa direction.

```

1 void determinerOrdre(Robot* robot, unsigned int caseSuivanteRobot,
  unsigned int largeurCircuit, Ordre* ordre1, Ordre* ordre2) {

```

```

2
3 unsigned int caseInitRobot = getCaseRobot(*robot);
4     tDirection dirRobot = getDirection(*robot);
5     unsigned int diff = caseSuivanteRobot - caseInitRobot;
6     tDirection nlleDirection;
7
8     if (diff == 1) { // Le switch a t remplacer par des ifs car on
9         nlleDirection = EST;
10    }
11    else if (diff == -1) {
12        nlleDirection = OUEST;
13    }
14    else if (diff == largeurCircuit) {
15        nlleDirection = SUD;
16    }
17    else if (diff == -largeurCircuit) {
18        nlleDirection = NORD;
19    }
20    else {
21        nlleDirection = dirRobot; // Pas de changement de direction
22    }
23    *ordre1 = ordreDuChangementDeDirection(dirRobot, nlleDirection);
24    setDirection(robot, nlleDirection);
25    *ordre2 = AV;
26    setCaseRobot(robot, caseSuivanteRobot);
27 }

```

Listing 4 – Fonction déterminer ordre

La fonction `resoudre_chemin_plus_court` permet de calculer le chemin optimal en testant toutes les permutations possibles :

```

1 Solution resoudre_chemin_plus_court(G_Graphe* g, unsigned int
2     point_obligatoires[3],
3     unsigned int depart, unsigned int
4     longueur_ville,
5     tDirection directionInitialeRobot)
6 {
7     Solution solution = {0};
8     solution.chemin_complet.nb_points = 0;
9     unsigned int meilleure_distance = 99999;
10
11     unsigned int point_cles[4];
12     point_cles[0] = depart;
13     for(unsigned int i = 1; i < 4; i++) {
14         point_cles[i] = point_obligatoires[i - 1];
15     }
16
17     unsigned int indices_classement[3] = {1, 2, 3};
18     unsigned int permutations[6][3];
19     permutation(indices_classement, permutations);
20
21     // Tester chaque permutation
22     for (unsigned int p = 0; p < 6; p++) {
23         unsigned int parcours[5];
24         parcours[0] = 0;
25         parcours[1] = permutations[p][0];
26         parcours[2] = permutations[p][1];

```

```

24     parcours[3] = permutations[p][2];
25     parcours[4] = 0;
26
27     unsigned int distance_totale = 0;
28     unsigned int estValide = 1;
29     Chemin segments[4];
30
31     // CALCUL DES SEGMENTS AVEC EXCLUSION
32     for (unsigned int i = 0; i < 4; i++) {
33         unsigned int index_debut = parcours[i];
34         unsigned int index_fin = parcours[i + 1];
35         unsigned int sommet_depart = point_cles[index_debut];
36         unsigned int sommet_arrivee = point_cles[index_fin];
37
38         // Determiner le sommet a exclure
39         unsigned int sommets_a_eviter = 0;
40         unsigned int nb_a_eviter = 1;
41
42         if (i > 0) {
43             Chemin segment_precedent = segments[i - 1];
44             sommets_a_eviter = segment_precedent.points[
segment_precedent.nb_points - 2];
45             nb_a_eviter = 1;
46         } else {
47             if (directionInitialeRobot == SUD) {
48                 sommets_a_eviter = depart - longueur_ville;
49             }
50             else if (directionInitialeRobot == NORD) {
51                 sommets_a_eviter = depart + longueur_ville;
52             }
53             else if (directionInitialeRobot == EST) {
54                 sommets_a_eviter = depart - 1;
55             }
56             else {
57                 sommets_a_eviter = depart + 1;
58             }
59         }
60
61         // Calculer le chemin avec exclusions
62         segments[i] = dijkstra_avec_exclusions(g, sommet_depart,
sommet_arrivee,
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
        &sommets_a_eviter,
        nb_a_eviter);
        if (segments[i].nb_points == 0) {
            estValide = 0;
            break;
        }
        // Verifier les demi-tours internes
        if (!chemin_autorise(segments[i])) {
            estValide = 0;
            break;
        }
        distance_totale += (segments[i].nb_points - 1);
    }

```

```

79         // Verification des jonctions
80         if (estValide) {
81             for (unsigned int i = 0; i < 3; i++) {
82                 if (!verifier_jonction_segments(segments[i], segments[i
+ 1])) {
83                     estValide = 0;
84                     break;
85                 }
86             }
87         }
88
89         if (!estValide) {
90             continue;
91         }
92
93         // Si cette permutation est valide et meilleure
94         if (distance_totale < meilleure_distance) {
95             meilleure_distance = distance_totale;
96
97             for(unsigned int i = 0; i < 3; i++) {
98                 solution.ordre[i] = point_cles[permutations[p][i]];
99             }
100
101             // Reconstruction du chemin complet
102             unsigned int index = 0;
103
104             for(unsigned int i = 0; i < 4; i++) {
105                 unsigned int debut = (i == 0) ? 0 : 1;
106
107                 for(unsigned int j = debut; j < segments[i].nb_points; j
++ ) {
108                     if (index < NB_POINTS_MAX) {
109                         solution.chemin_complet.points[index] = segments
[i].points[j];
110                         index++;
111                     }
112                 }
113             }
114
115             solution.chemin_complet.nb_points = index;
116         }
117     }
118
119     return solution;
120 }

```

Listing 5 – Fonction de résolution du chemin optimal

3.2 Conclusion

3.2.1 Répartition du travail

Tâches	Jahnis	Mathieu	Alioune	Ylann	Ilan	Neil
Construction du robot	X	X	X	X	X	X
Fonctions moteur	X	X	X		X	
Fonctions capteurs		X			X	X
Fonctions LCD		X	X	X	X	
Tests unitaires électronique	X	X			X	
Débogage électronique		X			X	
Fonctions liées à la détermination du plus court chemin	X					
Fonctions liées à la création du graphe						X
Tests unitaires algorithme	X					X
Débogage algorithme	X					X
Mise en place du makefile et bashrc						X
Rédaction du rapport	X	X	X	X	X	X

TABLE 3 – Répartition des tâches

3.2.2 Conclusion du projet

En conclusion, ce projet nous a permis de mettre nos connaissances en pratique afin de développer un robot autonome capable se déplacer selon un parcours donné. Les choix des composants ainsi que le choix de l'algorithme pour trouver le plus court chemin nous a permis d'atteindre un résultat plutôt satisfaisant en terminant deuxième du classement lors de la soutenance. Pendant le projet, nous avons fait face à de nombreux défis et de nombreuses difficultés comme la gestion du capteur RGB pour la détection des pastilles des couleurs. Toutefois, nous avons su communiquer et nous entraider pour surmonter ces difficultés et améliorer du mieux que l'on pouvait notre robot.

Un point très important que nous souhaitons évoquer : les différences entre la théorie et la réalité. Effectivement, nous avons dès le début défini le nombre de composant que nous souhaitions utilisés ainsi que la manière dont nous souhaitions réaliser les choses. Néanmoins, nous avons été obligés de faire de nombreux ajustements après plusieurs tests, tant dans la partie électrique du robot, que dans la partie résolution du parcours.

Finalement, ce projet nous a appris à travailler sur plusieurs aspects différents d'un même projet en répartissant le travail selon les préférences et forces de chacun. L'utilisation de Git a d'ailleurs été indispensable pour coordonner ce projet efficacement. Au terme de cette expérience, nous avons pu nous préparer aux attentes des entreprises et ce, malgré le délai assez court. Elle nous a permis de découvrir des méthodes de travail efficace et d'appliquer concrètement nos compétences théoriques.



FIGURE 5 – Groupe 15 du PI-smart robot

3.2.3 Commit Git

Voici le nombre de commit des membres du groupe :

- Ilan et Mathieu : 40 commit environ (travail en duo sur l'électronique)
- Jahnis : 55 commit environ
- Neil : 40 commit environ
- Alioune et Ylann : 15 commit environ (assistance la résolution de problèmes)
- Fonction `defiler(f: File): EtatDuDijkstra`
- Procédure `liberer_file(E f: File)`

4 Bibliographie

Références

- [1] Adafruit, *TCS34725 Color Sensor Datasheet*. <https://cdn-shop.adafruit.com/datasheets/TCS34725.pdf>
- [2] Powertech, *Capteur de suivi de ligne infrarouge TCRT5000*. <https://powertech-dz.org/product/capteur-de-suivi-de-ligne-infrarouge-tcrt5000-ir-1-4-5-8-canaux/>
- [3] Texas Instruments, *L293D Motor Driver Datasheet*. https://moodle.insa-rouen.fr/pluginfile.php/191118/mod_resource/content/3/l293d.pdf
- [4] Adafruit, *LCD1602 Datasheet*. <https://cdn-shop.adafruit.com/datasheets/TC1602A-01T.pdf>
- [5] Valeurs moyennes issues de plusieurs sources en ligne (absence de documentation officielle).

- [6] Philips, *PCF8574 I/O Expander Datasheet*. <https://www.alldatasheet.fr/datasheet-pdf/view/424340/PHILIPS/PCF8574T.html>
- [7] Raspberry Pi Foundation, *Raspberry Pi 4 Datasheet*. <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf>